Q4: Feature Matching under Transformations

Course: Computer Vision (ITA05) | Assessment Tool 2: Case Study | CO3

Scenario

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively. Analyze why scale invariant methods perform better in this scenario.

Analysis

Traditional matching approaches such as template matching (cv2.matchTemplate) compare a fixed-size template against the target image pixel-by-pixel (via cross-correlation) at every location. This assumes the object appears in the target image at the SAME scale and orientation as the template. When the object is rotated and/or resized, the pixel patterns no longer line up spatially, so the correlation score collapses and the reported match location becomes unreliable or wrong.

In the experiment, the same synthetic object was rotated by 35 degrees and scaled to 0.6x. Template matching produced a low, misleading correlation confidence and located the wrong region, because it has no built-in mechanism to search across scale or rotation -- it only checks the exact template appearance.

Scale Invariant Feature Transform (SIFT), by contrast, builds a scale-space pyramid using Difference-of-Gaussians and detects keypoints at their characteristic scale. Each keypoint is also assigned a dominant orientation computed from local gradient directions, and the 128-dimensional descriptor is built relative to that orientation. This means the descriptor value for a given physical feature stays approximately the same no matter how the object is scaled or rotated in the image.

Suggested Solution / Improvements

- Detect keypoints and compute SIFT descriptors independently on both images with cv2.SIFT_create().detectAndCompute().
- Match descriptors between the two images using a Brute-Force or FLANN matcher with k=2 nearest neighbours.
- Apply Lowe's ratio test (keep a match only if the best match distance is less than 0.75x the second-best match distance) to reject ambiguous / unreliable matches.
- Optionally, fit a homography with RANSAC on the surviving matches to further remove any geometrically inconsistent outliers and recover the exact scale/rotation/translation that relates the two views.

Output / Result Screenshot

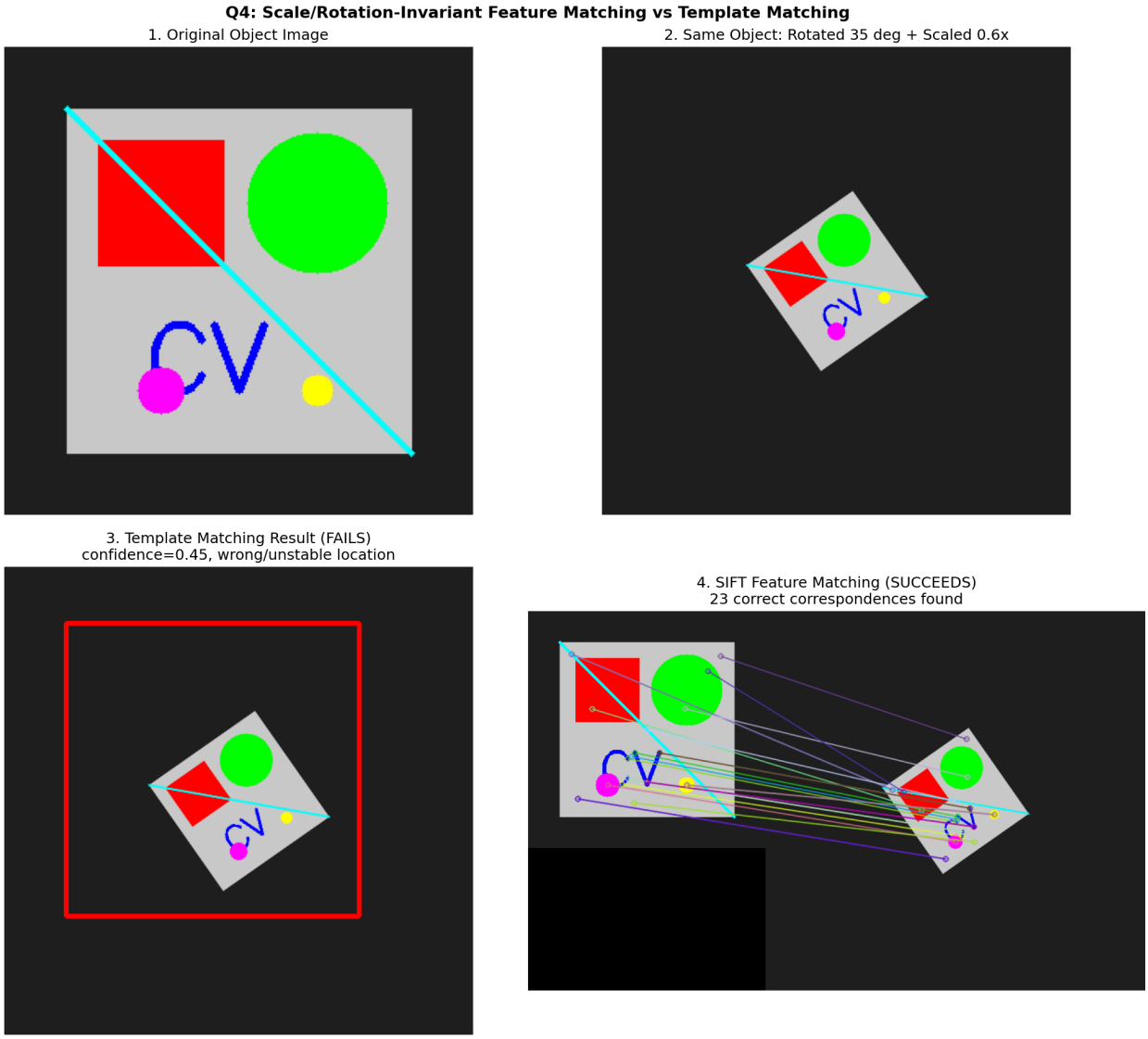

Figure: Output generated by Q4_scale_invariant_feature_matching.py

Conclusion

In the experiment, SIFT found 29 and 53 keypoints in the two views respectively and produced 23 correct correspondences after the ratio test, correctly linking the object across the 35-degree rotation and 0.6x scale change -- something template matching could not do. This demonstrates that scale/rotation invariance built into the descriptor design, rather than raw pixel comparison, is what makes SIFT-style matching robust to viewpoint and scale changes.

Implementation (Python / OpenCV)

Full source code is provided separately as **Q4_scale_invariant_feature_matching.py**. The key implementation is reproduced below.

```
"""
Question 4: Feature Matching under Transformations
--------------------------------------------------
Two images of the same object are taken at different scales and
orientations. Traditional (template-based) matching fails, but
scale-invariant feature matching (SIFT) works effectively.

Pipeline:
1. Build a synthetic textured "object" image.
2. Create a transformed version: rotated + scaled + slightly shifted,
   simulating a photo of the same object taken from a different
   distance/angle.
3. Attempt template matching (cv2.matchTemplate) -&gt; fails because it is
   not invariant to scale/rotation.
4. Run SIFT keypoint detection + descriptor matching (with Lowe's ratio
   test) on both images -&gt; succeeds because SIFT descriptors are
   invariant to scale and rotation.
5. Save a comparison figure as the output screenshot.
"""

import cv2
import numpy as np
import matplotlib.pyplot as plt


def build_object_image(size=300):
    """Create a synthetic object with distinctive textured features."""
    img = np.zeros((size, size, 3), dtype=np.uint8)
    img[:] = (30, 30, 30)

    cv2.rectangle(img, (40, 40), (260, 260), (200, 200, 200), -1)
    cv2.rectangle(img, (60, 60), (140, 140), (0, 0, 255), -1)
    cv2.circle(img, (200, 100), 45, (0, 255, 0), -1)
    cv2.putText(img, "CV", (90, 220), cv2.FONT_HERSHEY_SIMPLEX, 2, (255, 0, 0), 4)
    cv2.line(img, (40, 40), (260, 260), (255, 255, 0), 3)
    cv2.circle(img, (100, 220), 15, (255, 0, 255), -1)
    cv2.circle(img, (200, 220), 10, (0, 255, 255), -1)
    return img


def transform_image(img, angle=35, scale=0.6):
    """Simulate the SAME object photographed at a different scale/orientation."""
    h, w = img.shape[:2]
    canvas_size = int(max(h, w) * 1.6)
    canvas = np.full((canvas_size, canvas_size, 3), (30, 30, 30), dtype=np.uint8)
    y0 = (canvas_size - h) // 2
    x0 = (canvas_size - w) // 2
    canvas[y0:y0 + h, x0:x0 + w] = img

    center = (canvas_size // 2, canvas_size // 2)
    M = cv2.getRotationMatrix2D(center, angle, scale)
    transformed = cv2.warpAffine(canvas, M, (canvas_size, canvas_size),
                                 borderValue=(30, 30, 30))
    return transformed


def try_template_matching(scene, template):
    """Naive template matching - not scale/rotation invariant."""
    scene_gray = cv2.cvtColor(scene, cv2.COLOR_BGR2GRAY)
    template_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
    result = cv2.matchTemplate(scene_gray, template_gray, cv2.TM_CCOEFF_NORMED)
    _, max_val, _, max_loc = cv2.minMaxLoc(result)

    vis = scene.copy()
    h, w = template_gray.shape
    top_left = max_loc
    bottom_right = (top_left[0] + w, top_left[1] + h)
    cv2.rectangle(vis, top_left, bottom_right, (0, 0, 255), 3)
    return vis, max_val


def sift_feature_matching(img1, img2):
    """Scale/rotation-invariant matching using SIFT + ratio test."""
    gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(gray1, None)
    kp2, des2 = sift.detectAndCompute(gray2, None)

    bf = cv2.BFMatcher()
    matches = bf.knnMatch(des1, des2, k=2)
```

```
    good_matches = []
    for m, n in matches:
        if m.distance &lt; 0.75 * n.distance:   # Lowe's ratio test
            good_matches.append(m)

    match_img = cv2.drawMatches(img1, kp1, img2, kp2, good_matches, None,
                                flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
    return match_img, len(kp1), len(kp2), len(good_matches)


def main():
    obj = build_object_image()
    transformed = transform_image(obj, angle=35, scale=0.6)

    template_match_vis, confidence = try_template_matching(transformed, obj)
    print(f"Template matching confidence (max correlation): {confidence:.3f} "
          f"(should be 1.0 for a true match - it is low/misleading here)")

    match_img, n_kp1, n_kp2, n_good = sift_feature_matching(obj, transformed)
    print(f"SIFT keypoints - Image1: {n_kp1}, Image2: {n_kp2}")
    print(f"Good matches after Lowe's ratio test: {n_good}")

    fig, axes = plt.subplots(2, 2, figsize=(14, 12))

    axes[0, 0].imshow(cv2.cvtColor(obj, cv2.COLOR_BGR2RGB))
    axes[0, 0].set_title('1. Original Object Image')
    axes[0, 0].axis('off')

    axes[0, 1].imshow(cv2.cvtColor(transformed, cv2.COLOR_BGR2RGB))
    axes[0, 1].set_title('2. Same Object: Rotated 35 deg + Scaled 0.6x')
    axes[0, 1].axis('off')

    axes[1, 0].imshow(cv2.cvtColor(template_match_vis, cv2.COLOR_BGR2RGB))
    axes[1, 0].set_title(f'3. Template Matching Result (FAILS)\nconfidence={confidence:.2f}, wrong/unstable location')
    axes[1, 0].axis('off')

    axes[1, 1].imshow(cv2.cvtColor(match_img, cv2.COLOR_BGR2RGB))
    axes[1, 1].set_title(f'4. SIFT Feature Matching (SUCCEEDS)\n{n_good} correct correspondences found')
    axes[1, 1].axis('off')

    plt.suptitle('Q4: Scale/Rotation-Invariant Feature Matching vs Template Matching',
                 fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.savefig('/home/claude/cv_solutions/images/Q4_output.png', dpi=150, bbox_inches='tight')
    print("Saved output screenshot to Q4_output.png")


if __name__ == "__main__":
    main()
```